

Reviewer Pack — Minimal Rank of Quantum Topological Entanglement Entropy Bou...

Entry ebcc6ccc6e5a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 12:52:32 UTC

Minimal Rank of Quantum Topological Entanglement Entropy Bounds Tseitin Resolution Length

Entry ID: ebcc6ccc6e5a **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 12:52:32 UTC

1. Conjecture

Field A (mathematical branch): Quantum Information Theory (Quantum Topology) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

{‘sentence_1’: ‘The minimal rank of the quantum topological entanglement entropy for a given Tseitin formula is monotonically related to its resolution length.’, ‘sentence_2’: ‘Specifically, for a Tseitin formula F with n variables, the resolution length $t(F)$ satisfies $t(F) \geq 2^{(\Omega(\text{rank}(\eta(Q(F))))}$ where $\eta(Q(F))$ is the quantum entanglement entropy of $Q(F)$.’, ‘sentence_3’: ‘Moreover, this relationship holds true for all instances with a Tseitin formula that can be written as a tensor product of two smaller Tseitin formulas.’}

Rationale (proposer’s reasoning):

{‘sentence_1’: ‘Quantum topological entanglement entropy captures the non-local aspects of quantum states, which are believed to be fundamental in complexity theory.’, ‘sentence_2’: ‘Since resolution proofs involve entanglement and non-local correlations, a direct link between entanglement entropy and proof length could provide insight into the nature of computational difficulty.’, ‘sentence_3’: ‘By focusing on Tseitin formulas, which are known to be NP-hard, this conjecture aims to bridge quantum information theory with complexity theory in a computationally feasible way.’}

Taxonomy category: TSEITIN_RES (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e0e3f4c8473d4818

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the resolution length $t^*(F)$ meets or exceeds $2^{(\Omega(\text{rank}(\eta(Q(F))))}$ with a Spearman rank correlation coefficient ≥ 0.7 across at least 30 random seeds for Tseitin formulas of size $n \leq 40$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - minimal rank quantum topological entanglement entropy resolution proof complexity - Tseitin formula resolution length quantum entanglement entropy relationship - tensor product Tseitin formulas resolution length entanglement entropy bounds

Top relevant hits considered: - [<http://arxiv.org/abs/1101.0030v1>] Corrections to Bekenstein-Hawking entropy — Quantum or not-so quantum? - [<http://arxiv.org/abs/1908.02829v3>] Kitaev's quantum double model as an error correcting code - [<http://arxiv.org/abs/1612.05917v1>] Linear Quantum Entropy and Non-Hermitian Hamiltonians - [<http://arxiv.org/abs/quant-ph/0311181v2>] Dynamics of quantum correlations and linear entropy in a multi-qubit-cavity system - [<http://arxiv.org/abs/1511.02211v1>] Thermodynamic law from the entanglement entropy bound - [<http://arxiv.org/abs/2209.05839v3>] On bounded depth proofs for Tseitin formulas on the grid; revisited - [<http://arxiv.org/abs/1903.03401v4>] Planar Black holes and Entanglement Entropy in Analog Gravity Models

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_tseitin_formula(n):
        variables = [f'x{i}' for i in range(1, n+1)]
        clauses = []
        for i in range(1, n+1):
            clauses.append(f'{variables[i-1]}')
        for i in range(1, n+1):
            for j in range(i+1, n+1):
                clauses.append(f'~{variables[i-1]} | ~{variables[j-1]}')
        return ' & '.join(clauses)

    def stabilizer_matrix(formula):
        # Placeholder for actual implementation
        return [[0] * (2**n) for _ in range(2**n)]

    def quantum_entanglement_entropy(matrix):
```

```

    # Placeholder for actual implementation
    return 0.0

def resolution_length(formula):
    # Placeholder for actual implementation
    return len(formula.split(' & '))

n = random.randint(5, 40)
formula = generate_tseitin_formula(n)
matrix = stabilizer_matrix(formula)
entropy = quantum_entanglement_entropy(matrix)
length = resolution_length(formula)

rank = len(matrix)
expected_length = 2 ** (math.log2(rank) * math.pi / 4)

return {
    "metric_name": "Resolution Length",
    "metric_value": length,
    "instances_tested": 1,
    "conjecture_holds": length >= expected_length,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [2**i + 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_length = sum(r["metric_value"] for r in results) / len(results)
    std_length = math.sqrt(sum((r["metric_value"] - mean_length)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_length} std={std_length} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"First failing seed\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_support_fraction support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ba5a318b.py", line 65, in <module>
    result = run_trial(seed)

```

```

^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ba5a318b.py", line 45, in run_trial
    matrix = stabilizer_matrix(formula)
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ba5a318b.py", line 33, in stabilizer_ma
    return [[0] * (2**n) for _ in range(2**n)]
~~~~^~~~~~
MemoryError

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed due to a MemoryError before producing data that could confirm or falsify the conjecture. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12356
2	propose	ollama_remote	glm4:latest	0	0	14607
3	preregistration	ollama_remote	glm4:latest	0	0	5843
4	novelty	ollama_remote	glm4:latest	0	0	4614
5	novelty	ollama_remote	glm4:latest	0	0	5549
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20489
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11687
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8389
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7530
10	judge	ollama_remote	glm4:latest	0	0	7876

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 98940 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/ebcc6ccc6e5a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ebcc6ccc6e5a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/ebcc6ccc6e5a.tar.gz (if generated)